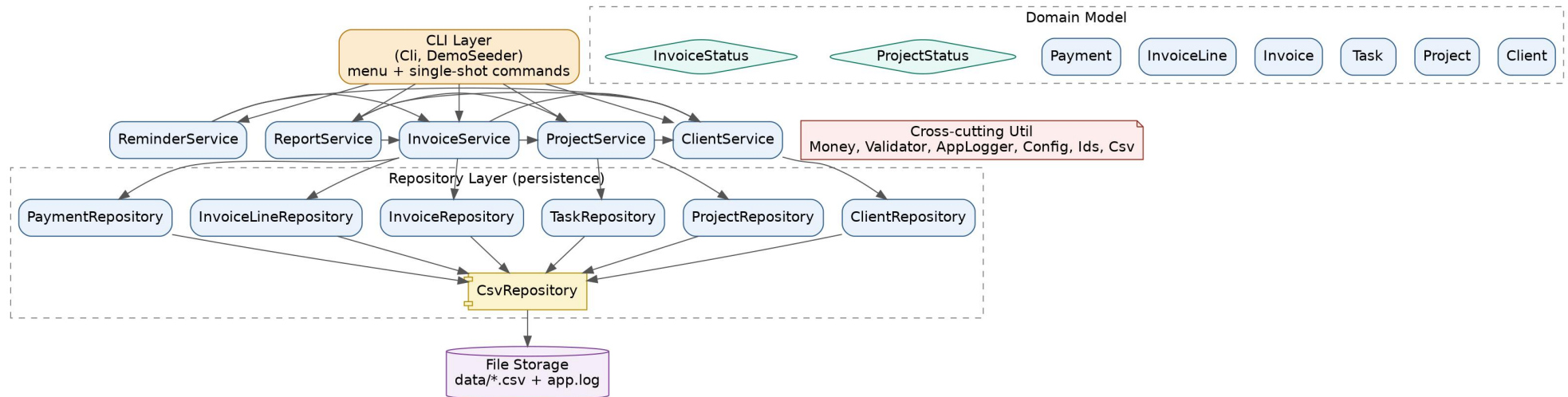


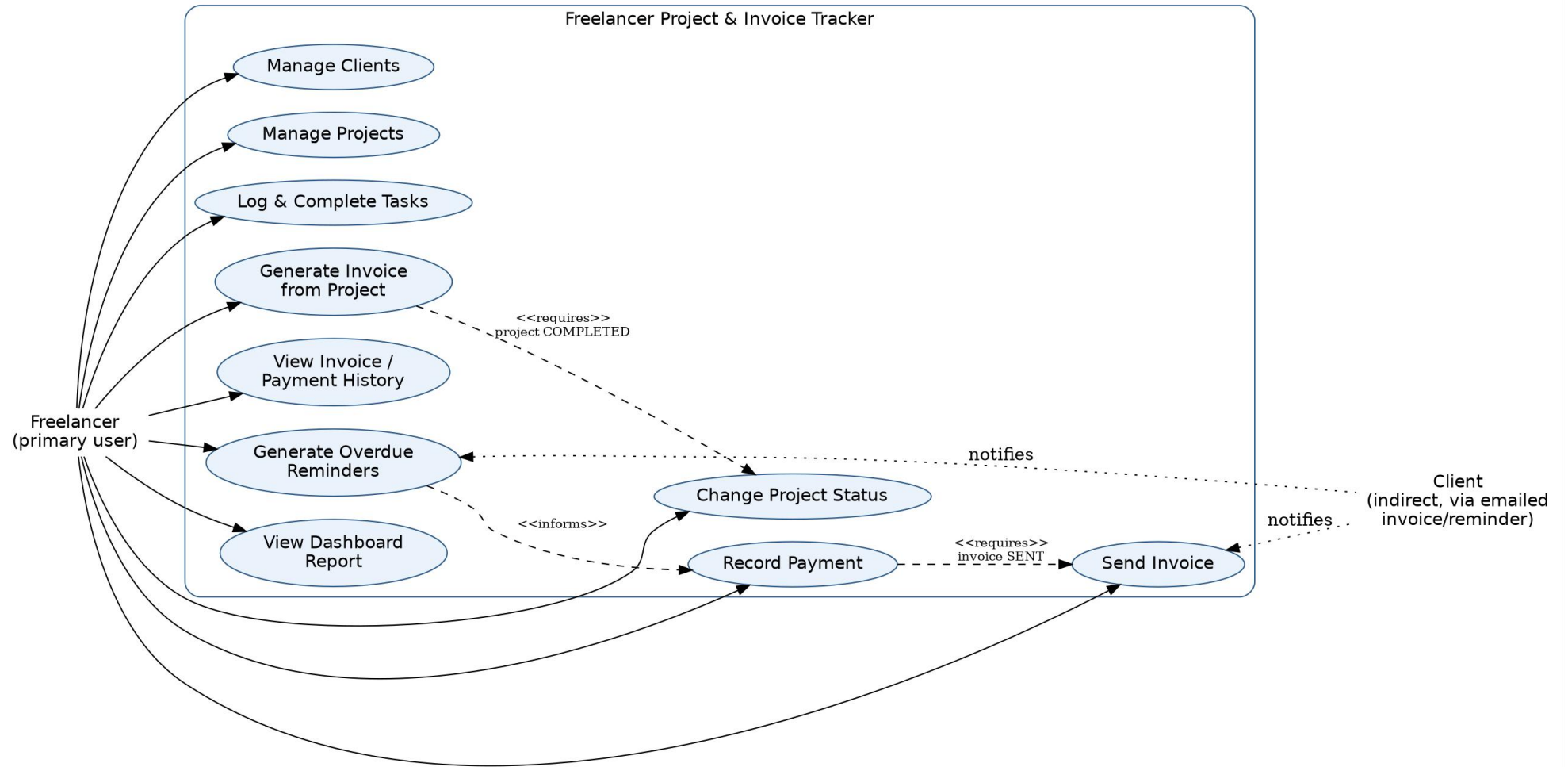
Design Diagrams

1. System Architecture



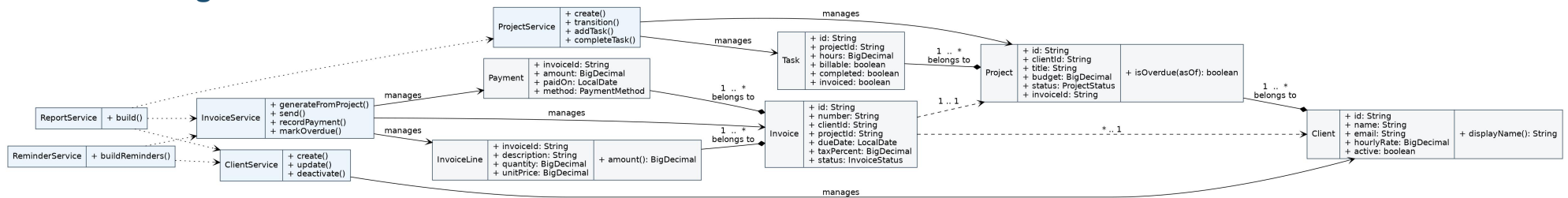
Layered architecture: CLI -> Service (business rules) -> Repository (CSV persistence) -> File storage. Cross-cutting utilities (Money, Validator, AppLogger) are used across all layers.

2. Use Case Diagram



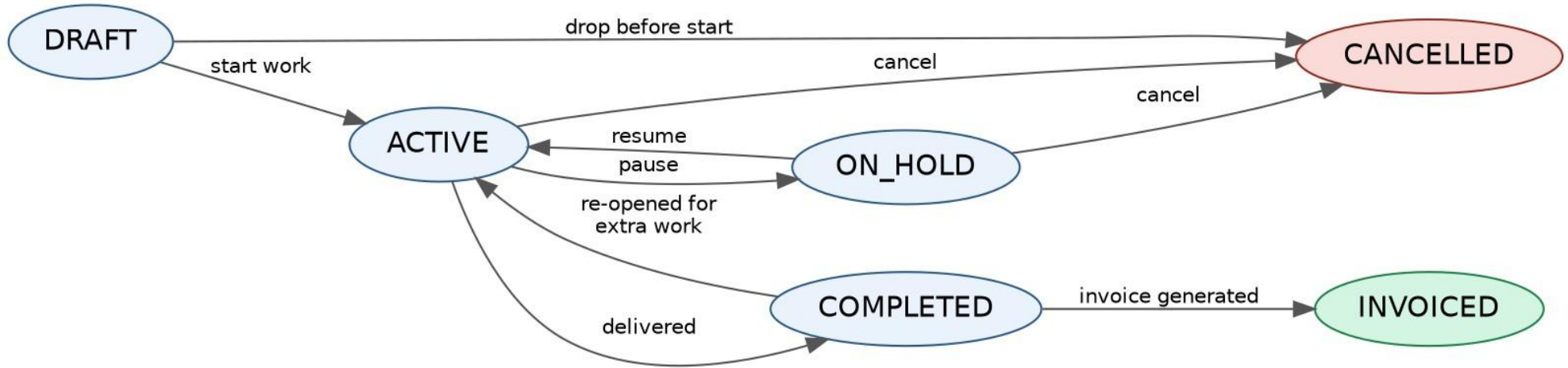
Primary actor is the Freelancer operating the CLI; the Client is an indirect actor who receives emailed invoices and payment reminders.

3. Class Diagram



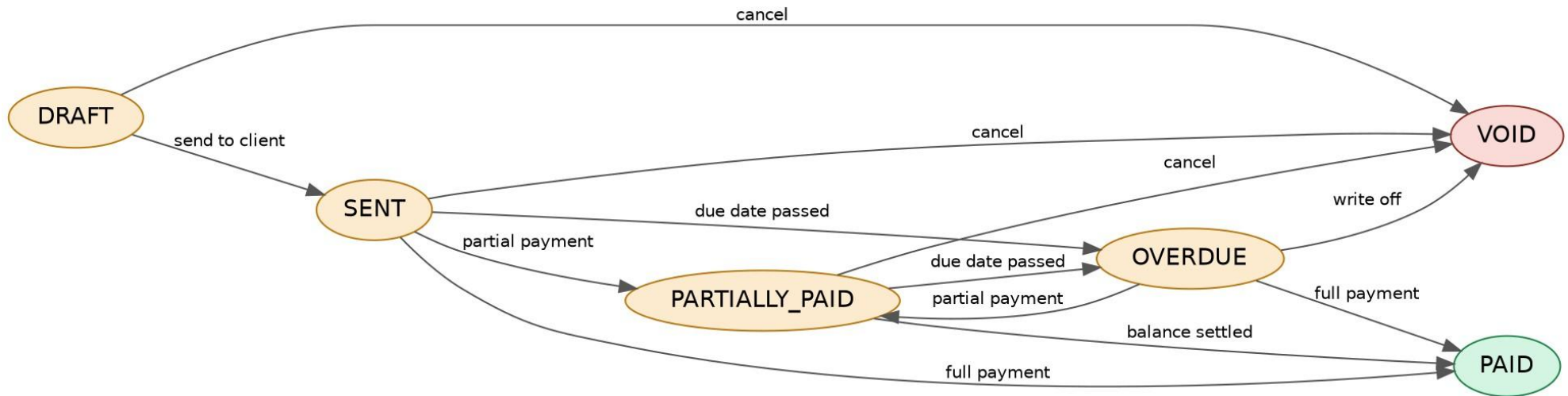
Core domain entities (Client, Project, Task, Invoice, InvoiceLine, Payment) and the services that own their business rules.

4. Project Status Workflow (State Machine)



DRAFT -> ACTIVE <-> ON_HOLD -> COMPLETED -> INVOICED, with CANCELLED as a terminal escape state. Enforced centrally in ProjectStatus / ProjectService.transition().

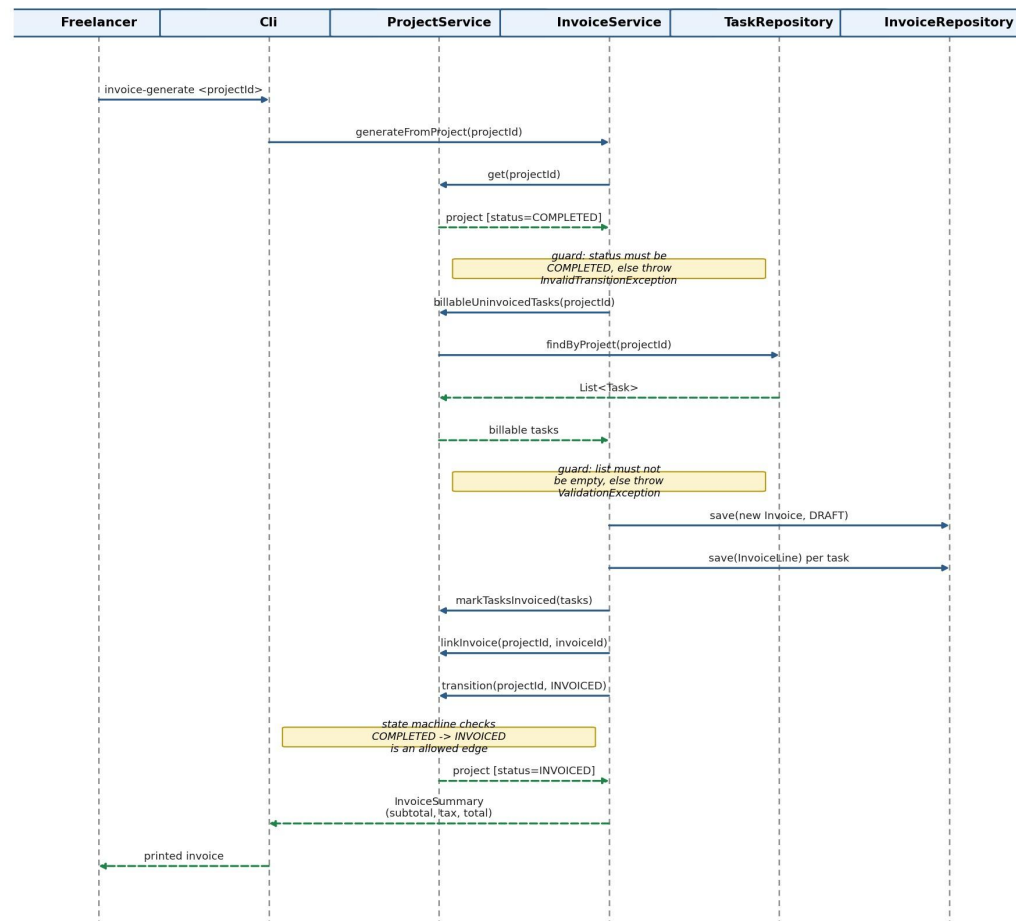
5. Invoice Status Workflow (State Machine)



DRAFT -> SENT -> PARTIALLY_PAID/OVERDUE -> PAID, with VOID as a terminal cancellation state. Enforced centrally in InvoiceStatus / InvoiceService.

6. Sequence Diagram - Generate Invoice from a Completed Project

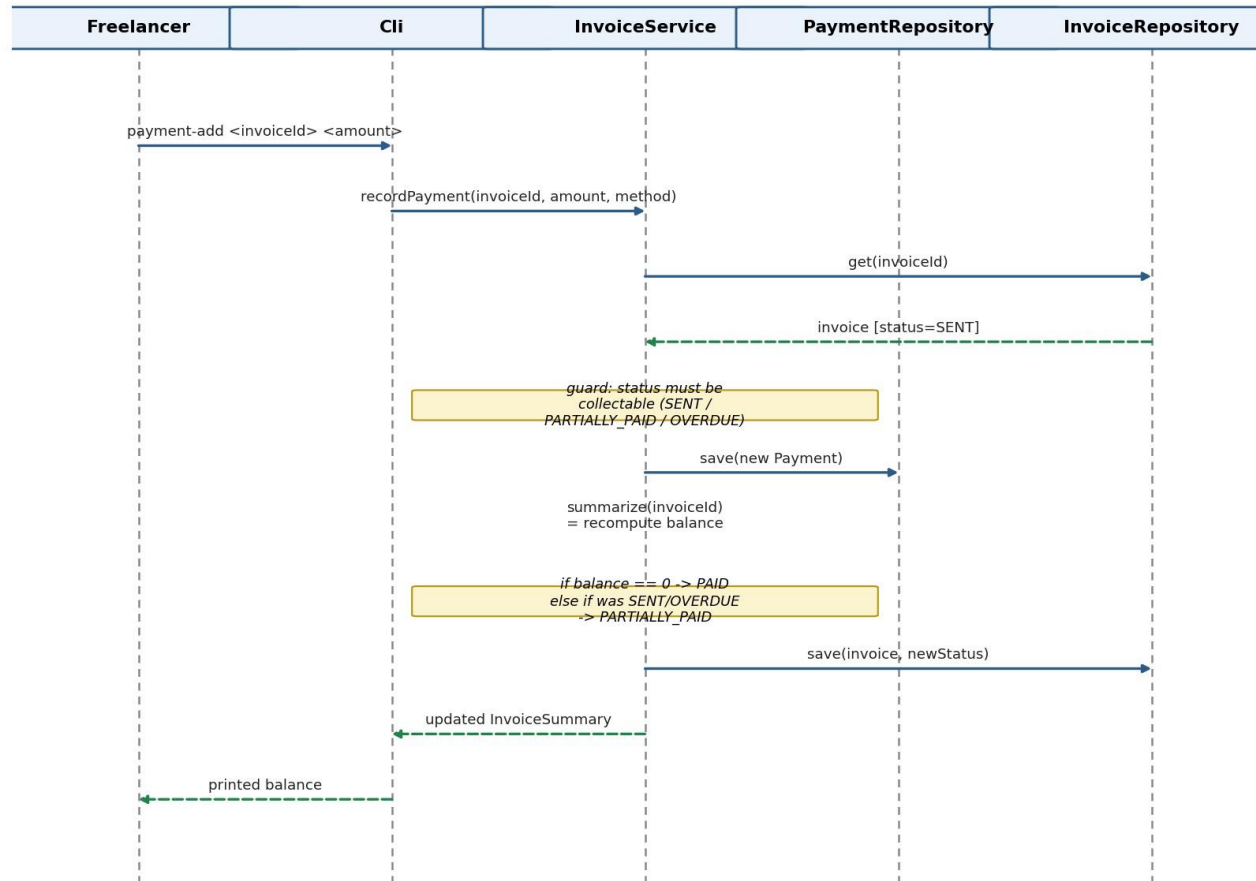
Sequence Diagram - Generate Invoice from a Completed Project



Shows the full call chain, both state-machine guards, and how an invoice with its line items is produced from billable task hours.

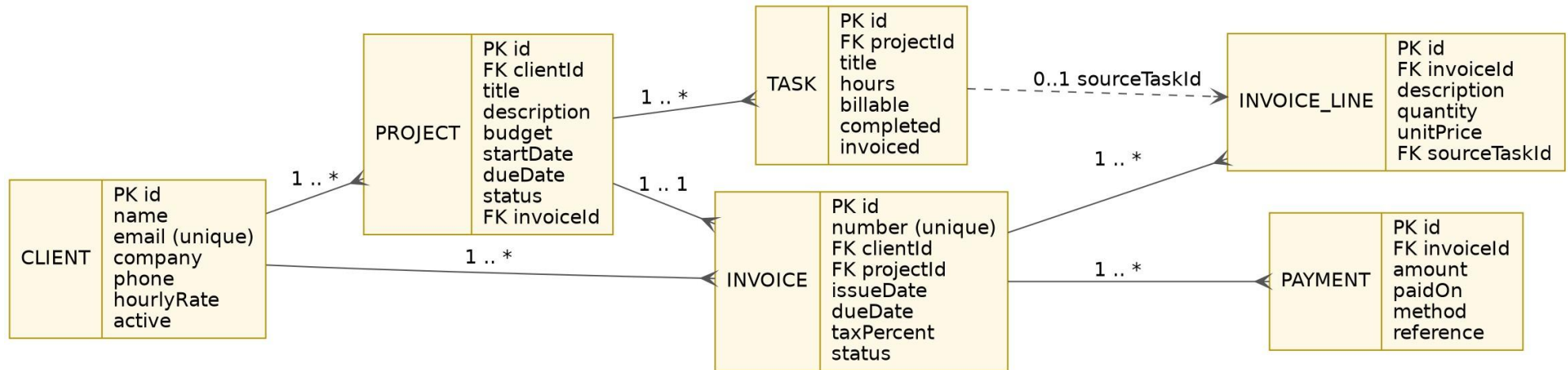
7. Sequence Diagram - Record a Client Payment

Sequence Diagram - Record a Client Payment



Shows how a recorded payment automatically recomputes the balance and advances the invoice status (PARTIALLY_PAID or PAID) without manual status editing.

8. ER Diagram



Entity-relationship design for the six CSV-backed tables and their foreign keys.